Aggregation

```
class Employee {

    String name;

    Employee(String name) {
        this.name = name;
    }

    void work() {
        System.out.println(name + " is working");
    }
    void isAlive()
    {
      System.out.println("alive........");
    }
}

class Company {

    String companyName;
    Employee employee;   // HAS-A relationship

    Company(String companyName, Employee employee) {
        this.companyName = companyName;
        this.employee = employee;
    }
}

public class EmployeeMain {

    public static void main(String[] args) {

        // Employee can exist independently
        Employee e = new Employee("Ravi");

        e.work();

        // Company HAS-A Employee
        Company c = new Company("ABC Technologies", e);

        //c.employee.work();

        System.out.println("Removing Employee from Company");
        c.employee = null;


        e.isAlive();
```

```
    }
}
```

Composition

Example 1

```
class Engine {

    void start() {
        System.out.println("Engine is running");
    }
    void isAlive()
    {
     System.out.println("Car is alive");
    }
}

class Car {

    Engine engine;   // default access

    Car() {
        engine = new Engine();   // Car creates Engine
    }

    void startCar() {

        engine.start();

        System.out.println("Car is running");
    }

    void removeEngine() {
        engine = null;   // Remove the Engine reference
    }
}

public class CarMain {

    public static void main(String[] args) {

        Car car = new Car();

        // Car has an Engine
```

```
        car.startCar();

        // Remove the Engine
        car.removeEngine();

        // Try to start the Car without Engine
        car.startCar();

        //car.engine.isAlive();
    }
}
```

Example 2

```
class Heart {

    void pumpBlood() {
        System.out.println("Heart is pumping blood");
    }
}

class Human {

    Heart heart;   // default access

    Human() {
        heart = new Heart();
    }

    void live() {

        heart.pumpBlood();

        System.out.println("Human is alive");
    }

    void removeHeart() {
        heart = null;   // Remove the Heart reference
    }
}

public class HumanMain {

    public static void main(String[] args) {

        Human human = new Human();

        // Human has a Heart
```

```
        human.live();

        // Remove the Heart
        human.removeHeart();

        // Try to live without Heart
        human.live();
    }
}
```